eitkit

Electrical Impedance Tomography Reconstruction Toolkit

Technical note

Abd'gafar Tunde Tiamiyu
School of Mathematics, Jilin University
Changchun 130012, Jilin, China
abdgafartunde@yahoo.com

Version 0.2.0, October 2026

DOI: 10.5281/zenodo.21037312

Summary

`eitkit` is an open-source Python package for two-dimensional difference electrical impedance tomography (EIT) on the disc. It generates triangular meshes, simulates electrode voltages with piecewise-linear finite elements and a point (gap) electrode model, computes the Jacobian with the adjoint method, and reconstructs conductivity changes with Tikhonov regularization (with an L-curve parameter choice) and total-variation regularization (with ADMM). This note documents the model and every algorithm as implemented, maps each step to the function that performs it, reports the verification results (convergence to a closed-form solution, finite-difference and reciprocity checks, golden-output tests), gives practical guidance on choosing the mesh size and the regularization parameters, and states the limitations. A step-by-step tutorial is provided separately as `examples/00_getting_started.ipynb`.

Contents

1 Introduction **2**

2 The forward problem **2**
2.1 Governing equation . . . 2
2.2 Point (gap) electrode model . . . 3
2.3 Mesh generation . . . 3
2.4 Finite element discretization . . . 3
2.5 Measurement protocol . . . 4

3 Difference imaging and the Jacobian **4**
3.1 Linearization . . . 4
3.2 The adjoint formula . . . 4
3.3 What the Jacobian tells you . . . 5

4 Reconstruction **5**
4.1 Tikhonov regularization . . . 5
4.2 Choosing λ: the L-curve . . . 5
4.3 Total-variation regularization . . . 5

5 Verification **6**
5.1 Convergence to the exact solution 6
5.2 Jacobian and reciprocity 6
5.3 Automated tests 7

6 Numerical example **7**

7 Choosing the parameters **10**

8 Software reference **10**

9 Limitations **11**

1 Introduction

Electrical impedance tomography recovers the conductivity σ inside a body from voltages measured on its boundary while currents are injected through electrodes. The inverse problem is severely ill-posed: in the worst case the conductivity depends on the boundary data only logarithmically [9], and reconstructions must be stabilized by regularization [2, 4]. *Difference* EIT avoids the need for an accurate absolute model by imaging the change between two states, for example two breathing phases, from the change in the measured voltages.

`eitkit` implements the standard linearized difference-imaging pipeline for a circular domain:

1. generate a mesh of the disc with electrodes at prescribed angles (`eitkit.mesh`);
2. choose the drive and measurement protocol (`eitkit.protocol`);
3. simulate difference voltages and compute the Jacobian (`eitkit.forward`);
4. reconstruct the conductivity change with Tikhonov or total-variation regularization (`eitkit.inverse`);
5. build phantoms and plot results (`eitkit.utils`).

It is intended for teaching, for prototyping reconstruction methods, and as a small, readable reference implementation: it depends only on NumPy, SciPy, and Matplotlib. It is not a replacement for full-featured packages such as EIDORS [1]; Section 9 lists what it does not do.

Installation. From the repository root, `pip install -e ".[dev]"` installs the package with the test and notebook dependencies; `uv sync -extra dev` reproduces the exact environment recorded in `uv.lock`. The tutorial notebook `examples/00_getting_started.ipynb` walks through a complete experiment and is executed by continuous integration on every change.

2 The forward problem

2.1 Governing equation

Let Ω be the disc of radius R (the unit disc by default) and $\sigma \in L^\infty(\Omega)$ the conductivity, bounded below by a positive constant. In the absence of interior sources the electric potential u satisfies

$$-\nabla \cdot (\sigma \nabla u) = 0 \quad \text{in } \Omega. \tag{1}$$

Current I is injected at electrode e^+ and extracted at electrode e^-. The potential is determined up to an additive constant; measured voltages are differences of potentials, so the constant does not matter.

2.2 Point (gap) electrode model

`eitkit` models each electrode as a single boundary point x_ℓ. The boundary condition is

$$\sigma \frac{\partial u}{\partial n} = I\left(\delta_{x_{e^+}} - \delta_{x_{e^-}}\right) \quad \text{on } \partial\Omega, \tag{2}$$

and a voltage measurement between electrodes a and b is $V = u(x_a) - u(x_b)$. This is the simplest electrode model. It ignores the electrode width and the contact impedance, which the complete electrode model [13] includes. Near a point source the potential has a logarithmic singularity, so the voltage at a driven electrode is not defined by (2); the measurement protocol (Section 2.5) therefore never uses a driven electrode.

For the uniform unit disc ($\sigma \equiv 1$, $R = 1$) the solution is known in closed form,

$$u(x) = -\frac{I}{\pi}\Big(\ln|x - x_{e^+}| - \ln|x - x_{e^-}|\Big) + C, \tag{3}$$

because $\partial_n \ln|x - y| = \frac{1}{2}$ on the unit circle for every boundary point $x \neq y$, so the normal flux vanishes away from the electrodes. Section 5 uses (3) to verify the solver.

2.3 Mesh generation

`make_circle_mesh(n_electrodes, h0, radius, max_iter, seed)` builds a triangulation with target edge length h_0 by a node-relaxation method in the spirit of DistMesh [11]:

1. *Boundary nodes.* $N_b = L\lceil 2\pi R/(Lh_0)\rceil$ nodes are placed on the circle at equal angles, where L is the number of electrodes. Boundary edges are therefore no longer than h_0, and because N_b is a multiple of L, every electrode angle $2\pi\ell/L$ is a boundary node. These nodes never move.
2. *Interior seed.* A hexagonal grid with spacing h_0 fills the disc; seeds closer than $0.6\,h_0$ to a boundary node are removed and the rest are jittered slightly (seeded by `seed`) to break Delaunay degeneracies.
3. *Relaxation.* Interior nodes are moved by repulsive edge forces $\max(h_0 - \ell, 0)$ for up to `max_iter` iterations, with a Delaunay retriangulation at each step; a node that leaves the disc is projected back onto the circle.
4. *Clean-up.* Triangles whose centroids lie outside the disc are removed, all triangles are oriented counterclockwise, and the boundary nodes are sorted by angle.

`place_electrodes(mesh, n_electrodes, start_angle)` returns, for each electrode, the boundary node closest in angle to $\theta_\ell =$ `start_angle` $+ 2\pi\ell/L$; with the default `start_angle = 0` this is exactly the node placed in step 1. Electrode 0 lies on the positive x-axis and the numbering runs counterclockwise.

Table 1 lists the resulting mesh sizes. Version 0.2.0 fixed a defect in earlier versions, in which only the electrode nodes were placed on the boundary: the boundary was then resolved by about 26 nodes regardless of h_0, the polygonal domain did not converge to the disc, and the electrode voltages did not converge under refinement.

2.4 Finite element discretization

The potential is approximated by continuous piecewise-linear (P1) functions on the mesh, and σ is constant on each triangle. For a triangle T_e with vertices (x_i, y_i), $i = 1, 2, 3$, area A_e, and conductivity σ_e, the gradient coefficients are $b_i = y_j - y_k$ and $c_i = x_k - x_j$ for the cyclic permutations (i, j, k), and the element stiffness matrix is

$$K^{(e)} = \frac{\sigma_e}{4A_e}\left(\mathbf{b}\mathbf{b}^\top + \mathbf{c}\mathbf{c}^\top\right) \in \mathbb{R}^{3\times 3}. \tag{4}$$

Table 1: Meshes from `make_circle_mesh` with $L = 16$ and `seed = 42`. The area ratio measures how well the polygon approximates the disc.

h_0	nodes	triangles	boundary nodes	area/π
0.20	107	180	32	0.99359
0.10	389	712	64	0.99839
0.08	604	1126	80	0.99897
0.05	1510	2890	128	0.99960
0.025	5907	11556	256	0.99990

`assemble_K(mesh, sigma)` sums these into the sparse global matrix K. The boundary condition (2) becomes the load vector f with $f_{e^+} = +I$, $f_{e^-} = -I$ at the electrode nodes and zeros elsewhere (`build_load_vector`). K is singular, with the constants in its kernel; the interior node nearest the center is grounded ($u = 0$) by symmetric row and column elimination (`pick_ground_node`, `apply_dirichlet_bc`). In `simulate` and `compute_jacobian` the modified matrix is factorized once with a sparse LU decomposition (`scipy.sparse.linalg.splu`) and reused for every right-hand side.

2.5 Measurement protocol

`adjacent_pattern(L)` returns the L drive pairs $(k, k+1 \bmod L)$. `measurement_pairs(L)` returns, for each drive step k, the $L-3$ adjacent pairs $(m, m+1 \bmod L)$ that contain neither driven electrode, as rows `[k, m, m+1]`. With $L = 16$ there are $16 \times 13 = 208$ measurements. By reciprocity, the voltage for drive pair (a, b) and measurement pair (c, d) equals the voltage for drive (c, d) and measurement (a, b), so only $L(L-3)/2 = 104$ measurements are independent. The Jacobian accordingly has rank 104 (Section 5).

3 Difference imaging and the Jacobian

3.1 Linearization

For a reference conductivity σ_0 (by default the constant `sigma0 = 1`), `simulate` returns the difference voltages

$$\delta V = V(\sigma) - V(\sigma_0) \in \mathbb{R}^P, \tag{5}$$

one entry per row of `meas_pairs`. The reconstruction methods use the linear model

$$\delta V \approx J\,\delta\sigma, \qquad \delta\sigma = \sigma - \sigma_0, \qquad J_{ie} = \frac{\partial V_i}{\partial \sigma_e}\Big|_{\sigma_0}, \tag{6}$$

with one row per measurement and one column per triangle. The relative linearization error $\|J\delta\sigma - \delta V\|/\|\delta V\|$ measures how far the data are from this model; for the two-inclusion phantom of Section 6, with contrasts 1.6 and 0.5 against a background of 1, it is 0.27.

3.2 The adjoint formula

Let u_k be the potential for drive step k and ϕ_i the potential generated by a unit current entering at the plus-electrode and leaving at the minus-electrode of measurement i. Then [5]

$$J_{ie} = -\int_{T_e} \nabla u_k \cdot \nabla \phi_i \, dx, \tag{7}$$

which for P1 elements reduces to

$$J_{ie} = -\frac{1}{4A_e}\Big[(\mathbf{b}\cdot\mathbf{u}_k^e)(\mathbf{b}\cdot\boldsymbol{\phi}_i^e) + (\mathbf{c}\cdot\mathbf{u}_k^e)(\mathbf{c}\cdot\boldsymbol{\phi}_i^e)\Big], \tag{8}$$

where $\mathbf{u}_k^e$ and $\boldsymbol{\phi}_i^e$ are the nodal values on the three vertices of T_e. The adjacent protocol uses at most L distinct measurement pairs, so `compute_jacobian` needs L forward and at most L adjoint solves, all with a single sparse factorization, rather than one adjoint solve per measurement; it assembles J in a few milliseconds (Section 6).

3.3 What the Jacobian tells you

The singular values of J decay quickly: at $h_0 = 0.08$ the 104 nonzero singular values span about 5×10^5. The column norms of J per unit area measure the sensitivity of the data to each region; they fall by more than an order of magnitude from the electrodes to the center, which is why EIT reconstructions are least resolved in the middle of the domain (see the sensitivity map in the tutorial).

4 Reconstruction

4.1 Tikhonov regularization

`tikhonov_solve(J, dV, lambda_, solver)` computes

$$\delta\sigma_\lambda = \arg\min_{\delta\sigma} \ \|J\delta\sigma - \delta V\|^2 + \lambda\|\delta\sigma\|^2 = (J^\top J + \lambda I)^{-1} J^\top \delta V. \tag{9}$$

The default `solver="direct"` solves the $E \times E$ normal equations with `numpy.linalg.solve`; `solver="lsqr"` applies LSQR to the equivalent augmented least-squares system, which avoids forming $J^\top J$.

4.2 Choosing λ: the L-curve

`choose_lambda(J, dV, lambdas, n_points, solver)` evaluates $x(\lambda) = \log_{10} \|J\delta\sigma_\lambda - \delta V\|$ and $y(\lambda) = \log_{10} \|\delta\sigma_\lambda\|$ for candidate values (by default 50 values log-spaced between $10^{-6}s$ and $10^{2}s$ with $s = \|J\|_F^2/P$) and returns the point of maximum signed curvature [6, 7]

$$\kappa = \frac{x'y'' - y'x''}{(x'^2 + y'^2)^{3/2}}, \tag{10}$$

with derivatives taken with respect to $t = \log_{10}\lambda$ by finite differences, along increasing λ. It also returns x and y for plotting.

Version 0.2.0 corrected this computation: earlier versions omitted the denominator of (10), which made the selected λ insensitive to the noise level (the same value at 30, 45, and 60 dB) and placed it on the over-smoothing branch of the curve.

The L-curve is a heuristic. In the example of Section 6 it selects $\lambda = 2.3 \times 10^{-9}$ (relative error 0.69), against an error of 0.58 for the best λ, which can only be found when the truth is known. The *discrepancy principle* [10], which chooses λ so that the residual $\|J\delta\sigma_\lambda - \delta V\|$ matches the noise norm, fails here. The clean data lie in the range of J (the part outside it is 1.6×10^{-14} of $\|\delta V\|$), so the linearization error is fitted as if it were signal and does not show up in the residual; meanwhile 71% of the noise lies outside the 104-dimensional range of J and can never be fitted. With the residual equal to the noise norm, the principle selects $\lambda \approx 9 \times 10^{-11}$, and the relative error is 1.9.

4.3 Total-variation regularization

`tv_solve(J, dV, alpha, mesh, rho, max_iter, tol, warm_start)` computes

$$\delta\sigma_\alpha = \arg\min_{\delta\sigma} \ \tfrac{1}{2}\|J\delta\sigma - \delta V\|^2 + \alpha\|D\delta\sigma\|_1, \tag{11}$$

where D (`build_gradient_op`) has one row per interior edge of the mesh and two nonzero entries, $+1$ and -1, for the two triangles that share that edge. Thus $\|D\delta\sigma\|_1$ is the sum of the absolute jumps of $\delta\sigma$ across interior edges. Total variation [12] weights each jump by the length of its edge; D does not, so on a quasi-uniform mesh $\|D\delta\sigma\|_1$ approximates $\mathrm{TV}(\delta\sigma)/\bar{h}$ for a typical edge length $\bar{h}$, and the appropriate α changes with h_0.

Problem (11) is solved by ADMM [3] with the splitting $z = D\delta\sigma$ (Algorithm 1). The $\delta\sigma$-update matrix $H = J^\top J + \rho D^\top D$ is symmetric positive definite and is factorized once, so each iteration costs one pair of triangular solves and a few sparse products. With `rho="auto"` the penalty is $\rho = \mathrm{tr}(J^\top J)/\mathrm{tr}(D^\top D)$, which balances the two terms of H. The iteration stops when both the primal residual r and the dual residual s are small (Boyd et al. [3], Section 3.3.1), with F the number of interior edges; the absolute tolerance (default 10^{-8}, in units of $\delta\sigma$) matters only when the solution is nearly constant. ADMM converges slowly, and more slowly for small α: in the example below, `tol = 1e-4` is met after about 1500 iterations for $\alpha = 2 \times 10^{-7}$, 3300 for 5×10^{-8}, and 24 000 for 10^{-8}. The default `max_iter` is 50 000, and `tv_solve` warns if it is reached first. A warm start initializes z; the dual variable restarts at zero, so it does not necessarily shorten the run.

Version 0.2.0 changed the stopping test. Earlier versions tested only the primal residual, with a default of 200 iterations, which stopped far from the minimizer: in the example, 300 iterations gave a relative error of 0.427 against 0.395 at convergence.

Algorithm 1 ADMM for TV-regularized difference EIT (`tv_solve`)

Require: J, δV, $\alpha > 0$, $\rho > 0$, tolerances ε (`tol`) and ε_a (`abs_tol`), starting point $\delta\sigma^0$ (zero, or `warm_start`)

factorize $H = J^\top J + \rho D^\top D$; set $z^0 = D\delta\sigma^0$, $u^0 = 0$, $\kappa = \alpha/\rho$
for $k = 0, 1, \ldots,$ `max_iter` $- 1$ **do**
  $\delta\sigma^{k+1} = H^{-1}\big(J^\top \delta V + \rho D^\top (z^k - u^k)\big)$
  $z^{k+1} = \mathcal{S}_\kappa\big(D\delta\sigma^{k+1} + u^k\big)$ ▷ $\mathcal{S}_\kappa(v) = \mathrm{sign}(v)\max(|v| - \kappa, 0)$
  $u^{k+1} = u^k + D\delta\sigma^{k+1} - z^{k+1}$
  $r = \|D\delta\sigma^{k+1} - z^{k+1}\|, \quad s = \rho\|D^\top (z^{k+1} - z^k)\|$
  if $r \le \sqrt{F}\varepsilon_a + \varepsilon \max(\|D\delta\sigma^{k+1}\|, \|z^{k+1}\|)$ **and** $s \le \rho\big(\sqrt{E}\varepsilon_a + \varepsilon\|D^\top u^{k+1}\|\big)$ **then stop**
  end if
end for
return $\delta\sigma^{k+1}$

5 Verification

5.1 Convergence to the exact solution

`scripts/verify_convergence.py` compares the FEM electrode voltages with the closed form (3) on the uniform unit disc, for every drive step of the adjacent protocol and all 13 measurements per step, on meshes with h_0 from 0.2 to 0.025. Table 2 and Figure 1 show the mean relative error. It decreases at every level. The meshes are not nested, so the rate between consecutive levels varies (from 0.9 to 3.6); a least-squares fit of log error against $\log h_0$ gives a slope of 2.6. Before the mesh fix of version 0.2.0, the same study gave errors of 0.30, 0.41, 0.25, 0.31, 0.10, 0.13, and 0.027 for the same seven meshes, rising at three of the six refinements.

5.2 Jacobian and reciprocity

- *Finite differences.* `scripts/verify_jacobian.py` compares 20 entries of the adjoint Jacobian with central finite differences of `simulate`; the largest relative difference is 2.4×10^{-7}.

Table 2: Mean relative error of the FEM electrode voltages against the exact solution (3), and the observed rate between consecutive meshes.

h_0	nodes	boundary nodes	relative error	rate
0.200	107	32	6.64×10^{-2}	–
0.140	205	48	2.94×10^{-2}	2.29
0.100	389	64	8.89×10^{-3}	3.55
0.070	787	96	3.62×10^{-3}	2.52
0.050	1510	128	1.52×10^{-3}	2.59
0.035	3051	192	5.68×10^{-4}	2.75
0.025	5907	256	4.16×10^{-4}	0.92

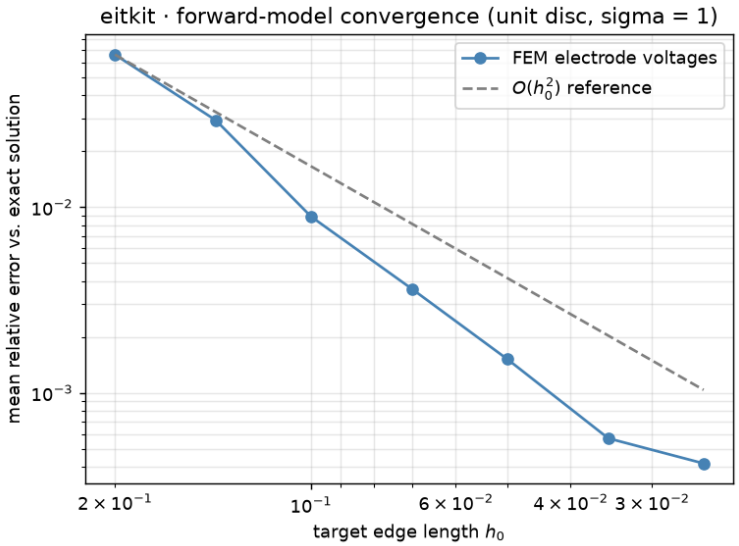


Figure 1: Convergence of the forward model (Table 2), with an $O(h_0^2)$ reference line.

- *Reciprocity.* Swapping drive and measurement pairs changes the computed voltage by at most 1.3×10^{-15} (relative) on an inhomogeneous conductivity, as the symmetry of K requires.
- *Rank.* J has numerical rank $104 = L(L-3)/2$ at $h_0 = 0.1$, 0.08, and 0.05, as reciprocity predicts.

5.3 Automated tests

The test suite (185 tests, `pytest`) checks mesh validity and quality, the protocol, the forward solver and Jacobian, both reconstruction methods, the L-curve corner (including its dependence on the noise level), the phantoms and plotting helpers, the ADMM stopping test (including a constant solution) and warm start, boundary resolution and forward convergence against (3), and a golden-output test that reruns the demo notebook's pipeline and compares mesh counts, voltages, and reconstruction errors with stored values (`tests/golden/demo_notebook.json`). Continuous integration runs the suite on Python 3.10 to 3.13, runs every script from outside the repository, and executes both notebooks.

6 Numerical example

The example in `scripts/reconstruct_and_plot.py`, the demo notebook, and the tutorial uses $L = 16$ electrodes, $h_0 = 0.08$ (604 nodes, 1126 triangles), unit current, and a background

conductivity of 1 S/m with a disc of 1.6 S/m and an ellipse of 0.5 S/m (Figure 2).

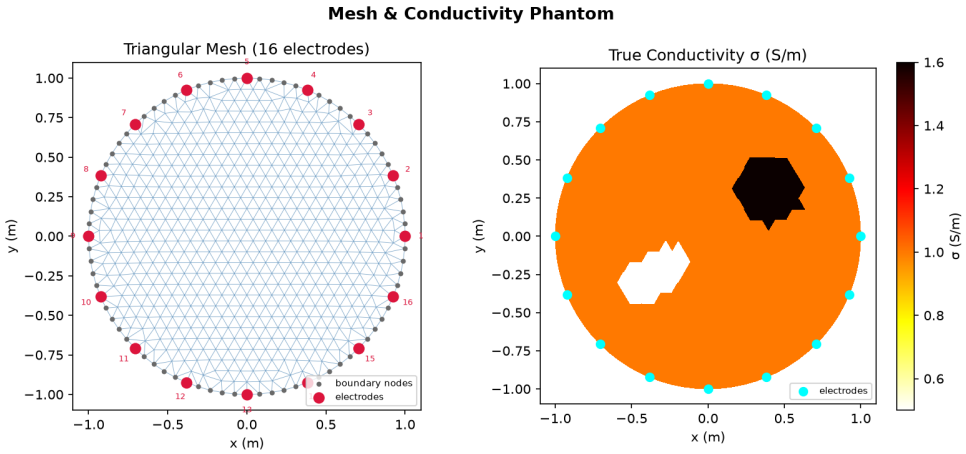


Figure 2: Mesh with the 16 electrodes, and the true conductivity.

Data. White Gaussian noise at 45 dB SNR is added to the difference voltages (Figure 3); at this level the noise norm is about 0.5% of the signal norm.

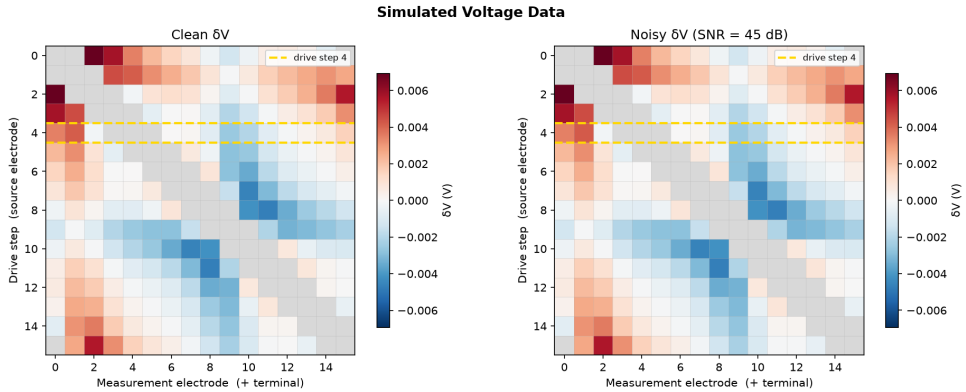


Figure 3: Clean and noisy difference voltages, arranged by drive step and measurement pair.

Regularization parameter. Figure 4 shows the L-curve and its corner, $\lambda = 2.3 \times 10^{-9}$. The reconstructions below use the fixed value $\lambda = 10^{-7}$, close to the best value 6.3×10^{-8} (Table 3).

Reconstructions. Figure 5 shows the Tikhonov and TV reconstructions, and Table 3 lists the relative errors $\|\delta\sigma - \delta\sigma_{\text{true}}\|/\|\delta\sigma_{\text{true}}\|$. TV recovers both inclusions with sharp boundaries and a flat background; both methods underestimate the contrast, because a single linearized step cannot reach the full contrast and the regularization pulls the solution towards zero. To avoid the *inverse crime* [8], the last column of Table 3 uses data simulated on a finer mesh ($h_0 = 0.04$, 4506 triangles) and reconstructed on the $h_0 = 0.08$ mesh. The errors hardly change (Tikhonov is even slightly better): since the mesh fix, the discretization error at $h_0 = 0.08$ (below 1% in the voltages) is small compared with the linearization and regularization errors.

Run times. Table 4 lists wall-clock times for three mesh sizes. The forward solves, the Jacobian, and the Tikhonov solve take milliseconds; mesh generation and the TV iterations take seconds.

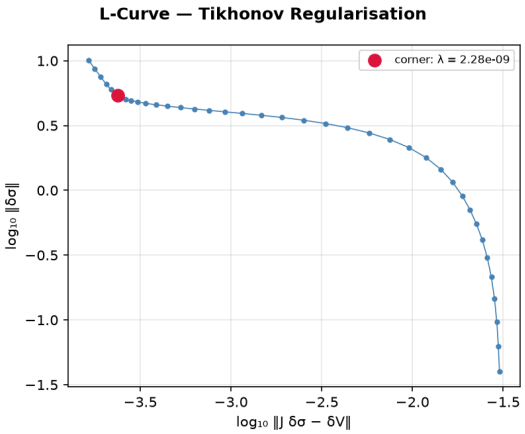


Figure 4: L-curve and the corner returned by `choose_lambda`.

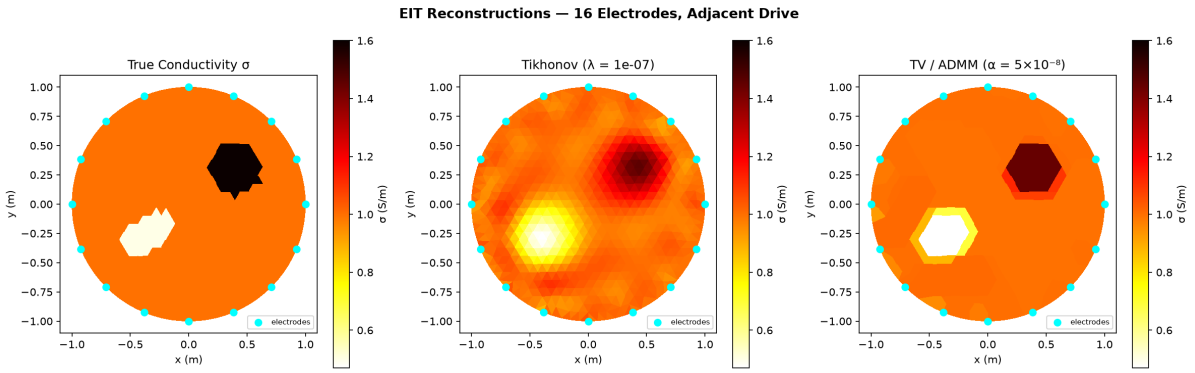


Figure 5: True conductivity, Tikhonov reconstruction ($\lambda = 10^{-7}$), and TV reconstruction ($\alpha = 5 \times 10^{-8}$).

Table 3: Relative reconstruction errors for the example (45 dB SNR).

method	same-mesh data	fine-mesh data
Tikhonov, $\lambda = 10^{-7}$ (fixed)	0.579	–
Tikhonov, $\lambda = 6.3 \times 10^{-8}$ (best)	0.575	–
Tikhonov, $\lambda = 2.3 \times 10^{-9}$ (L-curve)	0.689	0.659
TV, $\alpha = 5 \times 10^{-8}$	0.395	0.396

Table 4: Wall-clock times (Apple M-series laptop, Python 3.13) for a single-disc phantom; TV is run to convergence with the default tolerances (4912, 2051, and 1554 iterations).

h_0	triangles	mesh	`simulate` + `compute_jacobian`	Tikhonov	TV
0.10	712	0.48 s	0.006 s	0.003 s	0.93 s
0.08	1126	0.82 s	0.006 s	0.008 s	0.77 s
0.05	2890	2.31 s	0.016 s	0.083 s	3.85 s

7 Choosing the parameters

Mesh size h_0. The forward error is about 0.9% at $h_0 = 0.1$ and 0.15% at $h_0 = 0.05$ (Table 2), which is well below the noise and linearization errors of typical experiments. The number of triangles E grows like h_0^{-2}; the Jacobian costs $O(PE)$ memory and the direct Tikhonov solve up to $O(E^3)$ operations, so h_0 between 0.05 and 0.1 is a good default. For honest tests, simulate data on a finer mesh than the one used for reconstruction.

Tikhonov parameter λ. The scale of λ follows the scale of $J^\top J$, which depends on the current, the domain, and the mesh. Start from the L-curve corner, plot the reconstruction for λ ten times smaller and larger, and prefer the largest λ that keeps the features you expect. For the example ($I = 1$, unit disc, $h_0 = 0.08$), good values lie between 10^{-9} and 10^{-7}.

TV parameter α. In the example, with ADMM run to convergence, the relative error is 0.436, 0.404, 0.395, 0.419, and 0.470 for $\alpha = 10^{-8}$, 2×10^{-8}, 5×10^{-8}, 10^{-7}, and 2×10^{-7}, so values between 3×10^{-8} and 5×10^{-8} are best. Because D is not weighted by edge length, retune α when you change h_0. Small values of α need many more ADMM iterations; if `tv_solve` warns that it reached `max_iter`, increase it rather than accept the result.

Noise and contrast. `add_noise(dV, snr_db, rng)` adds white Gaussian noise at a prescribed SNR over the whole vector. Real systems also have systematic errors (electrode positions, contact impedance) that the model does not include; treat the SNR as an upper bound on data quality. Large contrasts increase the linearization error and make the reconstructed contrast too small.

8 Software reference

Table 5 lists the public functions.

Conventions. Conductivities are per triangle, in S/m; currents in A; voltages in V; lengths in units of the radius. Arrays are NumPy arrays: `sigma` has shape $(E,)$, `drive_pairs` $(L, 2)$, `meas_pairs` $(P, 3)$ with columns [drive step, plus electrode, minus electrode], `dV` $(P,)$, and J (P, E). Electrode 0 is at angle `start_angle` (default 0) and the numbering is counterclockwise.

Reproducing the results. All figures in this note are in `figures/` and are regenerated by `python scripts/reconstruct_and_plot.py` and `python scripts/verify_convergence.py -outdir figures`. The scripts run from any directory and accept `-outdir` and `-show`. The golden values are regenerated by `python tests/test_golden.py`.

Table 5: Public functions (module prefixes relative to `eitkit`).

function	purpose
`mesh.make_circle_mesh(n_electrodes, h0, radius, max_iter, seed)`	triangular mesh of the disc (`Mesh`)
`mesh.place_electrodes(mesh, n_electrodes, start_angle)`	electrode nodes (`ElectrodeConfig`)
`protocol.adjacent_pattern(L)`	drive pairs
`protocol.measurement_pairs(L)`	measurement schedule
`protocol.add_noise(voltages, snr_db, rng)`	white Gaussian noise
`forward.assemble_K(mesh, sigma)`	global stiffness matrix
`forward.simulate(mesh, ec, sigma, drive, meas, current, sigma0)`	difference voltages δV
`forward.compute_jacobian(mesh, ec, sigma, drive, meas, current)`	Jacobian J (adjoint method)
`inverse.tikhonov_solve(J, dV, lambda_, solver)`	Tikhonov reconstruction (9)
`inverse.choose_lambda(J, dV, lambdas, n_points, solver)`	L-curve corner and curve
`inverse.build_gradient_op(mesh)`	edge-jump operator D
`inverse.tv_solve(J, dV, alpha, mesh, rho, max_iter, tol, warm_start)`	TV reconstruction (11)
`utils.make_phantom(mesh, inclusions, sigma_background)`	piecewise-constant phantom (circle, ellipse, rectangle, ring, triangle)
`utils.plot_mesh`, `plot_conductivity`, `plot_voltages`	plotting helpers

9 Limitations

- *Geometry.* Two-dimensional discs only; real bodies are three dimensional, and currents spread out of the electrode plane.
- *Electrodes.* Point electrodes without contact impedance; the complete electrode model [13] is not implemented, and voltages at driven electrodes cannot be used.
- *Protocol.* Adjacent drive and adjacent measurement only.
- *Imaging.* Linearized difference imaging with a single step; there is no iterative (Gauss–Newton) reconstruction and no absolute imaging.
- *Scale.* J and $J^\top J$ are dense, so memory grows like E^2; meshes with more than a few thousand triangles become slow.
- *ADMM.* Convergence is slow for small α (tens of thousands of iterations); no accelerated or preconditioned variant is implemented.
- *TV operator.* Edge jumps are not weighted by edge length, so α depends on the mesh size.
- *Uncertainty.* No uncertainty quantification.

References

[1] A. Adler and W.R.B. Lionheart. Uses and abuses of EIDORS: an extensible software base for EIT. *Physiological Measurement*, 27(5):S25–S42, 2006.

[2] L. Borcea. Electrical impedance tomography. *Inverse Problems*, 18(6):R99–R136, 2002.

[3] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1):1–122, 2011.

[4] M. Cheney, D. Isaacson, and J.C. Newell. Electrical impedance tomography. *SIAM Review*, 41(1):85–101, 1999.

[5] D.B. Geselowitz. An application of electrocardiographic lead theory to impedance plethysmography. *IEEE Transactions on Biomedical Engineering*, 18(1):38–41, 1971.

[6] P.C. Hansen. Analysis of discrete ill-posed problems by means of the L-curve. *SIAM Review*, 34(4):561–580, 1992.

[7] P.C. Hansen and D.P. O'Leary. The use of the L-curve in the regularization of discrete ill-posed problems. *SIAM Journal on Scientific Computing*, 14(6):1487–1503, 1993.

[8] J. Kaipio and E. Somersalo. Statistical inverse problems: discretization, model reduction and inverse crimes. *Journal of Computational and Applied Mathematics*, 198(2):493–504, 2007.

[9] N. Mandache. Exponential instability in an inverse problem for the Schrödinger equation. *Inverse Problems*, 17(5):1435–1444, 2001.

[10] V.A. Morozov. On the solution of functional equations by the method of regularization. *Soviet Mathematics Doklady*, 7:414–417, 1966.

[11] P.-O. Persson and G. Strang. A simple mesh generator in MATLAB. *SIAM Review*, 46(2):329–345, 2004.

[12] L.I. Rudin, S. Osher, and E. Fatemi. Nonlinear total variation based noise removal algorithms. *Physica D*, 60(1–4):259–268, 1992.

[13] E. Somersalo, M. Cheney, and D. Isaacson. Existence and uniqueness for electrode models for electric current computed tomography. *SIAM Journal on Applied Mathematics*, 52(4):1023–1040, 1992.